UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

Instituto de Ciencias Básicas e Ingeniería

Licenciatura en Ciencias Computacionales

Práctica 5

Análisis léxico de elementos de un lenguaje de programación

Alumno: Daniela Lopez Martinez

Catedrático: EDUARDO CORNEJO VELÁZQUEZ

Grupo: 6-3

Asignatura: AUTOMATAS Y COMPILADORES



1. Identificación

Institución	Universidad Autónoma del Estado de Hidalgo (UAEH)
Nombre de la práctica	Análisis léxico de elementos de un lenguaje de programación
Número de práctica	5
Alumna	Daniela López Martínez
Semestre y grupo	6° semestre, grupo 3
Profesor	Eduardo Cornejo
Fecha de realización	25 de octubre de 2026

2. Introducción

El análisis léxico constituye la primera fase de un compilador. Su función principal es transformar una secuencia de caracteres del código fuente en componentes léxicos individuales denominados **tokens**. Para realizar este proceso, el analizador compara los caracteres de entrada con patrones definidos mediante expresiones regulares.

Entre las tareas principales del análisis léxico se encuentran:

- Leer los caracteres del archivo de entrada.
- Identificar y clasificar tokens como palabras reservadas, identificadores, operadores y constantes.
- Ignorar espacios y comentarios cuando no se requiere mostrarlos como tokens.
- Llevar el control de las líneas del archivo fuente.
- Detectar caracteres que no coinciden con ninguna regla definida.

3. Marco teórico

- **Categoría léxica:** clase a la que pertenece un elemento del lenguaje, por ejemplo, identificadores, operadores o constantes numéricas.
- **Token:** unidad reconocida por el analizador léxico que representa una categoría.
- **Lexema:** secuencia concreta de caracteres del código fuente que coincide con una regla.
- **Expresión regular:** patrón que describe un conjunto de cadenas que pueden reconocerse.
- **Flex/Lex:** herramienta que genera un analizador léxico a partir de expresiones regulares y acciones escritas en C.

En esta práctica se consideran palabras reservadas como `if`, `else`, `while`, `int` y `float`; identificadores como `contador`; operadores como `+`, `++`, `+=`, `&&` y `==`; constantes numéricas como `10` y `3.1416`; símbolos especiales como llaves, paréntesis y punto y coma; y comentarios de una o varias líneas.

4. Objetivos

Objetivo general

Desarrollar un programa en Flex/Lex que lea un archivo con código fuente similar a C++ e identifique y clasifique sus elementos mediante reglas de análisis léxico.

Objetivos específicos

- Definir expresiones regulares para reconocer palabras reservadas, identificadores, constantes, operadores y comentarios.
- Implementar un analizador léxico que procese un archivo de entrada.
- Probar el programa con diferentes fragmentos de código.
- Identificar el comportamiento del analizador ante caracteres no reconocidos.

5. Desarrollo

5.1. Autómatas finitos deterministas (AFD)

Las expresiones regulares de Flex permiten describir patrones que pueden representarse mediante autómatas finitos. A continuación se presentan dos ejemplos conceptuales.

AFD 1: Reconocimiento de identificadores

`Estado inicial - letra o _ ->Estado de aceptación`

`Estado de aceptación - letra, dígito o _ ->Estado de aceptación`

Un identificador comienza con una letra o guion bajo y puede continuar con letras, dígitos o guiones bajos.

AFD 2: Reconocimiento de números decimales

`Estado inicial - dígito ->Estado de entero`

`Estado de entero - punto ->Estado intermedio`

`Estado intermedio - dígito ->Estado de decimal`

El patrón decimal utilizado en esta práctica requiere dígitos antes y después del punto.

5.2. Código fuente del analizador léxico (`Analizador.l`)

El siguiente código define patrones para identificar los principales elementos léxicos. Se utiliza una regla para comentarios de una línea y otra para comentarios multilínea. Los espacios y tabuladores se ignoran; los saltos de línea actualizan el contador de líneas.

```
%{
#include <stdio.h>

int cant_comentarios = 0;
int num_linea = 1;
%}

%option noyywrap

DIGITO [0-9]
LETRA [a-zA-Z_]
```

```
ID {LETRA}({LETRA}|{DIGITO})*
ENTERO {DIGITO}+
DECIMAL {DIGITO}+"."{DIGITO}+
ESPACIO [ \t\r]+

%%

\n { num_linea++; }
{ESPACIO} { /* Ignorar espacios */ }

"//"[^\n]* {
                                cant_comentarios++;
                                printf("[LINEA %d] Token: COMENTARIO_LINEA | Lexema: %s\n",
                                      num_linea, yytext);
                            }

"/*"([^*]|\*+[^*/])*\*+"/" {
                                int i;
                                cant_comentarios++;
                                printf("[LINEA %d] Token: COMENTARIO_MULTILINEA | Lexema: %s\n",
                                      num_linea, yytext);
                                for (i = 0; yytext[i] != '\0'; i++) {
                                    if (yytext[i] == '\n') num_linea++;
                                }
                            }

"if"|"else"|"while"|"for"|"do"|"switch"|"case"|"break"|"continue"|"return"|"int"|"float"|"double"|"
    char"|"bool"|"void"|"class"|"struct"|"public"|"private"|"protected" {
                                printf("[LINEA %d] Token: PALABRA_RESERVADA | Lexema: %s\n",
                                      num_linea, yytext);
                            }

"&&"|"||"|"!"|"=="|"!="|"<="|">="|"<"|">" {
                                printf("[LINEA %d] Token: OPERADOR_LOGICO/RELACIONAL | Lexema: %s\n",
                                      num_linea, yytext);
                            }

"++"|"--"|"+="|"-="|"*="|"/="|"+"|"-"|"*"|"/"|"%" {
                                printf("[LINEA %d] Token: OPERADOR_ARITMETICO | Lexema: %s\n",
                                      num_linea, yytext);
                            }

"=" {
                                printf("[LINEA %d] Token: ASIGNACION | Lexema: %s\n",
                                      num_linea, yytext);
                            }

"{"|"}"|"["|"]"|"("|")"|";"|"," {
                                printf("[LINEA %d] Token: SIMBOLO_ESPECIAL | Lexema: %s\n",
                                      num_linea, yytext);
                            }

{DECIMAL} {
                                printf("[LINEA %d] Token: CONSTANTE_DECIMAL | Lexema: %s\n",
                                      num_linea, yytext);
                            }

{ENTERO} {
                                printf("[LINEA %d] Token: CONSTANTE_ENTERA | Lexema: %s\n",
                                      num_linea, yytext);
                            }

{ID} {
                                printf("[LINEA %d] Token: IDENTIFICADOR | Lexema: %s\n",
                                      num_linea, yytext);
                            }
```

```
77
78  . {
79                              printf("[ERROR LEXICO] Linea %d: Caracter no reconocido '%s'\n",
80                                     num_linea, yytext);
81                          }
82
83  %%
84
85  int main(int argc, char **argv) {
86      if (argc > 1) {
87          FILE *archivo = fopen(argv[1], "r");
88          if (!archivo) {
89              perror("Error al abrir el archivo de prueba");
90              return 1;
91          }
92          yyin = archivo;
93      } else {
94          fprintf(stderr, "Uso: %s <archivo_prueba.cpp>\n", argv[0]);
95          return 1;
96      }
97
98      yylex();
99
100     printf("\n========================================\n");
101     printf("RESUMEN: Total de comentarios detectados: %d\n", cant_comentarios);
102     printf("========================================\n");
103
104     return 0;
105 }
```

5.3. Compilación y ejecución

En una terminal con Flex y un compilador de C instalados, el procedimiento habitual es:

```
1 flex Analizador.l
2 gcc lex.yy.c -o Analizador
3 Analizador.exe prueba.cpp
```

6. Resultados y pruebas realizadas

6.1. Estructura de archivos en la carpeta de trabajo

En esta sección se debe mostrar la carpeta donde se encuentran el archivo de reglas, el ejecutable y los archivos de prueba.

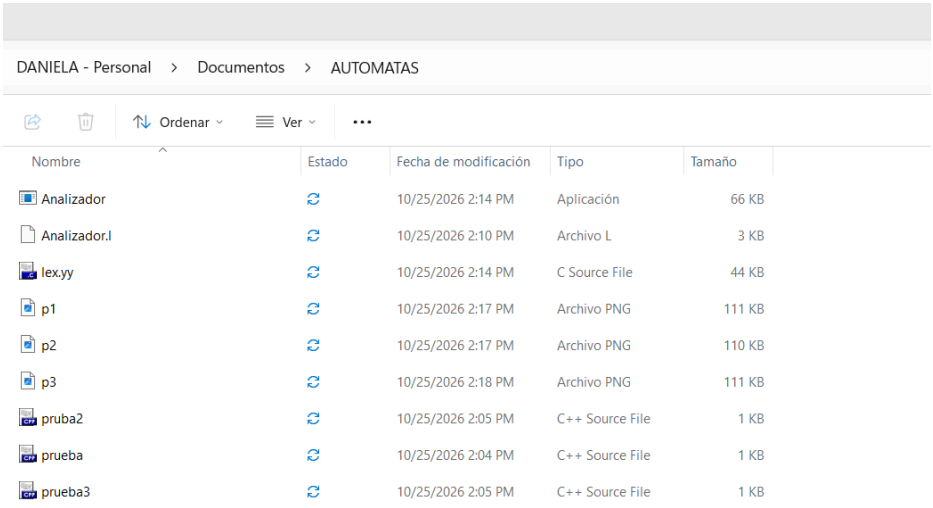


Figura 1: Archivos del proyecto y programas de prueba.

6.2. Ejecución de pruebas de entrada

a) Prueba 1: `prueba.cpp`

Esta prueba permite observar el reconocimiento de elementos básicos del lenguaje, como palabras reservadas, identificadores, constantes numéricas, símbolos y comentarios.

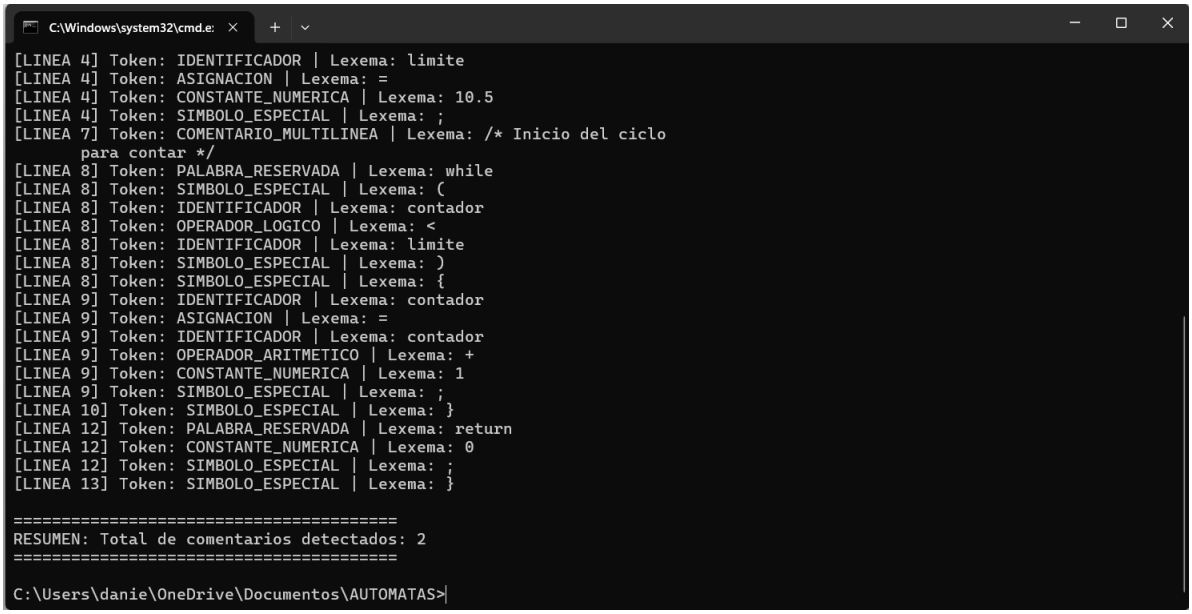


Figura 2: Resultado de ejecución de la primera prueba.

b) Prueba 2: `pruba2.cpp`

Esta prueba se enfoca en los operadores compuestos y lógicos, como ++, +=, && y !=.

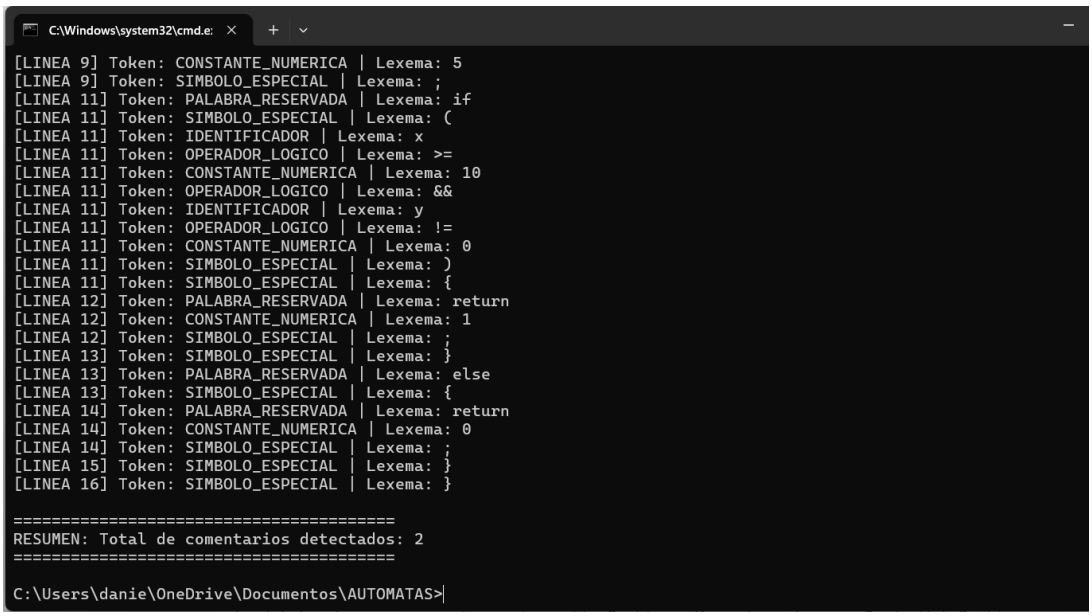


Figura 3: Resultado de ejecución de la segunda prueba.

c) Prueba 3: `prueba3.cpp`

Esta prueba utiliza caracteres no contemplados por las reglas, por ejemplo `$`, `@` y `~`, para comprobar la generación de mensajes de error léxico.

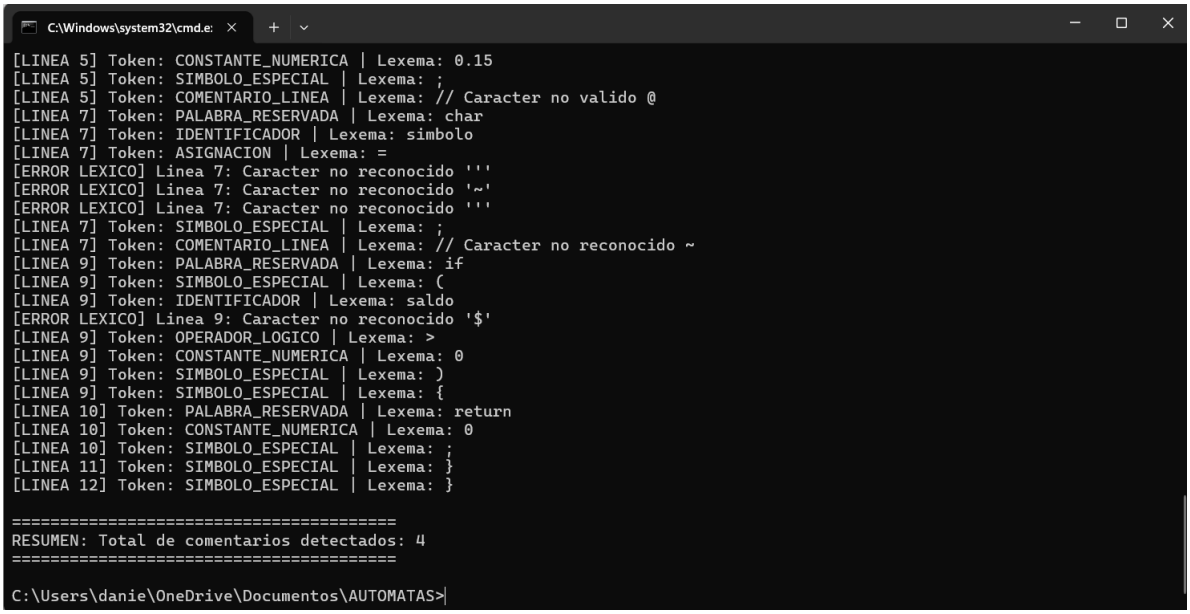


Figura 4: Resultado de la tercera prueba: detección de errores léxicos.

6.3. Análisis de resultados

Las pruebas permiten evaluar si el analizador identifica los lexemas de acuerdo con las reglas definidas. En la primera prueba se revisa el reconocimiento de elementos básicos; en la segunda, el tratamiento de operadores compuestos; y en la tercera, la detección de caracteres no reconocidos.

Cuando aparece un carácter que no coincide con las reglas anteriores, la regla final `.` muestra un mensaje de error y permite que el análisis continúe. El resultado debe comprobarse con las capturas obtenidas durante la ejecución real del programa.

7. Cuestionario

1. **¿Qué patrones se deben definir para identificar correctamente los tokens de números, operadores aritméticos y lógicos?**

   Para números enteros se puede utilizar `[0-9]+`; para decimales, `[0-9]+"."[0-9]+`. Los operadores se definen mediante alternativas, por ejemplo `` `+"|-"|`="|=" `` para operadores aritméticos y `"&&"|"||"|¡"|`="|¡="` para operadores lógicos o relacionales.

2. **¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos y reconocer los tokens?**

   Se abre el archivo de entrada con `fopen` y se asigna al flujo `yyin`. Después, `yylex()` lee el contenido, compara las cadenas con las reglas definidas y ejecuta la acción correspondiente cuando encuentra una coincidencia.

3. **¿Qué elementos de la herramienta Lex te permitieron implementar las reglas para el análisis léxico?**

   Se utilizaron definiciones de patrones como `DIGITO` e `ID`, los delimitadores `%%` para separar las secciones del archivo, la variable `yytext` para consultar el lexema reconocido y `yyin` para indicar

el archivo de entrada.

4. **¿Qué condiciones o recomendaciones se deben seguir al plantear las reglas para identificar los tokens?**

   Es importante definir patrones claros, colocar primero las reglas de operadores compuestos y considerar que Flex selecciona la coincidencia más larga. Cuando dos reglas reconocen la misma cantidad de caracteres, tiene prioridad la que aparece primero. Las palabras reservadas deben estar antes de la regla general de identificadores.

5. **¿Cómo se logró identificar errores léxicos y qué estrategias utiliza para recuperarse de ellos?**

   Se utilizó una regla final con el patrón `.`, que captura caracteres no reconocidos por las reglas anteriores. El programa imprime un mensaje con el número de línea y el carácter detectado, y continúa analizando el resto del archivo.

8. Conclusiones

El desarrollo de esta práctica permitió comprender la importancia del análisis léxico dentro del proceso de compilación. Mediante Flex/Lex se definieron expresiones regulares para reconocer distintos elementos de un programa y asociarles acciones en C. Las pruebas ayudan a verificar el reconocimiento de palabras reservadas, identificadores, operadores y constantes, así como la detección de caracteres no reconocidos. Esta práctica también permitió observar la utilidad de las expresiones regulares para construir analizadores léxicos de manera organizada.

9. Bibliografía

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2008). *Compilers: Principles, Techniques, and Tools* (2.ª ed.). Pearson.
- Levine, J. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media.
- GNU Project. (s. f.). *Flex: The Fast Lexical Analyzer*. https://westes.github.io/flex/manual/